Fortran program structure, `implicit none`, compiling and running

The idea

Python interprets your source line by line. Fortran is **compiled**: `gfortran` first translates the whole file into machine code, checking types on the way, and then you run the result. The extra step buys speed and errors caught *before* a 10-hour run starts, not in hour 9.

Anatomy of a program

```
program circle                        ! name of the program
  use, intrinsic :: iso_fortran_env, only: dp => real64
  implicit none                       ! every variable must be declared
  real(dp), parameter :: pi = 3.141592653589793_dp   ! a named constant
  real(dp) :: r, area                                ! variables
  r = 2.0_dp
  area = pi * r**2
  print *, 'area =', area
end program circle
```

The order is fixed: `program`, then `use` statements (modules; here the 64-bit real kind), then `implicit none`, then **all declarations**, then executable statements (a declaration after the first one is an error), then `end program`. `!` starts a comment, `**` is the power, and `print *, a, b` prints with a default format (nicer formatting is FT-1.4). Fortran is **case-insensitive**: `Q` and `q` are the same variable.

Types for now

`real(dp)` is a 64-bit real (a Python `float`), and `integer` is an integer. Write real literals with the `_dp` suffix: `2.0_dp`, `1.6e-13_dp`. A literal without it is *single* precision (about 7 digits), even when you assign it to a `real(dp)` variable. Kinds are FT-1.2; for today, just put `_dp` on every real number (forgetting it on `1.66053906660e-27` silently costs about 9 digits). A `parameter` is a constant fixed at compile time: assigning to it later is a compile error.

`implicit none`

Old Fortran had **implicit typing**: an undeclared name was created automatically, as an `integer` if its first letter was `i`–`n` and as a `real` otherwise. Any typo silently becomes a brand-new variable. `implicit none` switches this off, so every name must be declared; task 4 shows what that buys. Put it in every program unit you write.

Compiling and running

In the build command of task 1, `-Wall` turns on most warnings, `-fcheck=all` adds runtime checks (slower, for development only), and `-o solution` names the executable (default `a.out`); `./solution` runs it. Compiler messages point to a line and column: `solution.f90:12:16:` means line 12, column 16. Fix the **first** error first; later ones are often knock-on effects. Recompile after every edit: running an old binary is the classic "my fix did nothing" moment.